

★ WEEK 4 · PROBLEM SET

Finance Applications: Three Hands-On Problems

OLS Mini-Beta · Ridge Shrinkage · Lasso Selection

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real market data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public sources first — the prompts fetch free data; CSV files are offline fallbacks only

Problem	Primary source (public · no signup)	Provided fallback file
P1 · OLS mini-beta	yfinance — AAPL vs. SPY daily returns	fml_w4_panel_offline.csv
P2 · Ridge shrinkage	yfinance — the 10-stock panel from Lab 1	fml_w4_panel_offline.csv
P3 · Lasso selection	yfinance — 30 large caps + SPY (tracking)	fml_w4_panel_offline.csv
Hand calculations	Printed on the slides (4 points · 1 formula · 5 betas)	fml_w4_p1 / p3 hand-calc CSVs

Course rule: if data is free online (yfinance, FRED, OpenML), the prompt fetches it — files are for offline classrooms.

P1

PROBLEM 1 · OLS MINI-BETA

Fit the Line with Four Days of Data

One covariance, one variance, one division — a market beta by hand

Four Days of Market and Stock Moves

Fit $\hat{y} = \beta_0 + \beta_1 x$ by least squares. Is this a high-beta or low-beta stock?

	D1	D2	D3	D4
Market return x (%)	-2	0	2	4
Stock return y (%)	-3	0	2	5

Given

- Means: $\bar{x} = 1$, $\bar{y} = 1$.
- Everything else you compute yourself.

Your deliverables

- The demeaned columns, $\Sigma(x - \bar{x})(y - \bar{y})$ and $\Sigma(x - \bar{x})^2$.
- β_1 and β_0 , the fitted line, and all four residuals.
- One sentence: what does $\beta_1 = 1.30$ say about this stock on a down -2% market day?

The Tools: Slope = Co-movement over Variance

OLS in one dimension is just two sums and a division

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Reading the formula

- Top: how x and y move TOGETHER (co-movement).
- Bottom: how much x moves on its own (variance).
- β_0 just re-centers the line through the point $(\bar{x}, \bar{y})$.

Lab guide — four steps

- Step 1: subtract the means from each column.
- Step 2: multiply the pairs; sum $\rightarrow 26$. Square the x's; sum $\rightarrow 20$.
- Step 3: divide. Step 4: $\beta_0 = \bar{y} - \beta_1 \bar{x}$, then residuals.

The Whole Regression in One Table

Four rows of arithmetic — then read the residuals like a doctor

Day	x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$	$\hat{y} = -0.3 + 1.3x$	resid
D1	-2	-3	-3	-4	12	9	-2.9	-0.1
D2	0	0	-1	-1	1	1	-0.3	+0.3
D3	2	2	1	1	1	1	2.3	-0.3
D4	4	5	3	4	12	9	4.9	+0.1
Σ	—	—	0	0	26	20	—	SSE = 0.20

$$\hat{\beta}_1 = \frac{26}{20} = 1.30 \quad \hat{\beta}_0 = 1 - 1.30 \times 1 = -0.30 \quad \hat{y} = -0.30 + 1.30x$$

A High-Beta Stock, Read Aloud

$\beta_1 = 1.30$: when the market moves 1%, this stock moves 1.3% — in both directions

1.30

β_1 — the market beta;
amplifies every
market move

−0.30

β_0 — the drag when
the market is flat

0.20

SSE — the four
residuals barely miss;
the line fits

−2.9%

Prediction on a −2%
market day — risk you
can quote

Why finance loves this

- β is a sentence: 'this stock is 1.3× the market'.
- CAPM, hedging, risk budgets — all start from this division.

The Week 4 warning

- With 4 points and 1 clue, OLS is perfect.
- With 100 points and 400 clues, it grabs fakes — hence P2 and P3.

Scale It: A Real Market Beta

Paste the metaprompt — Claude Code fits the same line on five years of data

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) downloads 5 years of daily AAPL and SPY prices with  
   yfinance (auto_adjust=True) and computes returns;  
   (offline classroom: load fml_w4_panel_offline.csv,  
   using MKT as the market column)  
2) computes beta1 and beta0 by the hand formula  
   (demeaned sums) with numpy — no sklearn yet;  
3) verifies the same numbers with sklearn  
   LinearRegression and prints both side by side;  
4) plots the scatter of (market, stock) returns with  
   the fitted line, and prints R-squared;  
5) two-line verdict: is AAPL high- or low-beta, and  
   what does the R-squared say about noise (Week 1)?
```

[OUTPUT] What to expect

```
beta1 ~ 1.1 - 1.3 (varies  
  by window); beta0 ~ 0  
numpy and sklearn agree to  
  ~10 decimal places  
R-squared ~ 0.5 - 0.6:  
  half the daily move is  
  market, half is noise  
  
Your 4-day table, x 1,250.
```

P2

PROBLEM 2 • RIDGE

Watch the Budget Shrink the Beta

Same four days, one new knob — trace β as λ tightens

Put P1's Beta on a Budget

Same data ($\Sigma xy = 26$, $\Sigma x^2 = 20$). Compute the Ridge beta at $\lambda = 0, 5, 20, 80$

In one dimension Ridge has a closed form: the numerator stays, but λ joins the denominator. Trace the shrinkage path by hand.

Given (from P1)

- Co-movement $\Sigma(x-\bar{x})(y-\bar{y}) = 26$.
- Variance $\Sigma(x-\bar{x})^2 = 20$.
- OLS beta = $26/20 = 1.30$ — the $\lambda = 0$ anchor.

Your deliverables

- β_{ridge} at $\lambda = 0, 5, 20, 80$ (one division each).
- The prediction for a -2% market day at each λ .
- One sentence: where does β go as $\lambda \rightarrow \infty$, and what does the model become?

The Tool: One λ in the Denominator

Ridge = OLS with a weight on the scale — heavier λ , smaller β

$$\hat{\beta}_1^{\text{ridge}} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2 + \lambda} = \frac{26}{20 + \lambda}$$

Why it shrinks

- The numerator (evidence of co-movement) is fixed at 26.
- λ inflates the denominator — the evidence buys less beta.
- At $\lambda = 0$ you recover OLS exactly; nothing is free, nothing is lost.

Lab guide — per λ

- One division: $26 / (20 + \lambda)$.
- Then one prediction: $\hat{y}(-2) = \beta_0(\lambda) - 2\beta(\lambda)$, with $\beta_0(\lambda) = 1 - \beta(\lambda)$.
- Four λ values $\rightarrow$ four rows. That is the whole exercise.

The Shrinkage Path, Row by Row

Watch the beta melt from 1.30 toward zero as the budget tightens

λ	$\beta = 26 / (20 + \lambda)$	$\beta_0 = 1 - \beta$	$\hat{y}$ on a -2% market day	Reading
0	$26/20 = 1.30$	-0.30	-2.90%	plain OLS — full trust in the data
5	$26/25 = 1.04$	-0.04	-2.12%	mild budget — beta trimmed
20	$26/40 = 0.65$	+0.35	-0.95%	strict — half the sensitivity
80	$26/100 = 0.26$	+0.74	+0.22%	starved — barely reacts to the market

$$\lambda = 0: 1.30 \quad \lambda = 5: 1.04 \quad \lambda = 20: 0.65 \quad \lambda = 80: 0.26 \quad \longrightarrow \quad 0$$

From 1.30 to 0.26 — and Eventually to Zero

λ never changes the data; it changes how much the model trusts the data

1.30

$\lambda = 0$ — the OLS anchor; maximum trust

0.65

$\lambda = 20$ — half the beta with the same 4 points

→ 0

$\lambda \rightarrow \infty$ — β dies; the model predicts $\bar{y}$ for everyone

4

Data points — tiny samples are exactly where shrinkage helps

The bias-variance trade

- Shrinking β adds bias but removes variance (Week 1's formula).
- With 4 noisy points, a smaller β often predicts BETTER.

Note what never happened

- β shrank at every λ — but never hit exactly zero.
- Ridge calms clues; it cannot delete them. That is P3's job.

Scale It: Coefficient Paths on Ten Stocks

Paste the metaprompt — Claude Code draws the full shrinkage picture

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) downloads 5 years of daily returns for the 10-stock panel from class (AAPL, MSFT, GOOGL, AMZN, META, KO, PEP, XOM, JPM, JNJ) with yfinance; (offline classroom: load fml_w4_panel_offline.csv)
- 2) sets $y = \text{AAPL}$, $X =$ the other nine, standardized with a sklearn Pipeline (no test-set leakage);
- 3) sweeps Ridge alpha over a log grid from $1e-3$ to $1e3$ and records every coefficient at every alpha;
- 4) plots the nine coefficient paths on one chart ($x = \log \alpha$) and the validation MSE curve below;
- 5) prints the alpha at the validation minimum and a two-line verdict: which betas melt fastest, and does any path ever hit exactly zero (it should not).

[OUTPUT] What to expect

nine smooth curves melting
toward zero, never touching
validation MSE: a U-shape
with a clear minimum
correlated tech names shrink
together (the cousins)

Your 4-row table, drawn as
nine continuous paths.

P3

PROBLEM 3 • LASSO

Fire the Lucky Factor

Five zoo candidates, one threshold — selection by subtraction

A Five-Animal Factor Zoo

Standardized, uncorrelated clues with these OLS betas. Run Lasso at $\lambda = 0.25$ and $\lambda = 0.40$

Clue (factor)	OLS beta b
value	+0.90
momentum	+0.35
size	+0.10
quality	-0.60
'lucky' factor	+0.05

Your deliverables

- Lasso betas at $\lambda = 0.25$: subtract λ from each $|b|$, floor at 0, restore the sign.
- The same at $\lambda = 0.40$. Who survives each cut?
- One sentence: what would Ridge have done to the 'lucky' factor instead?

Because the clues are standardized and uncorrelated, Lasso reduces to this simple 'soft threshold'.

The Tool: Soft Thresholding

Subtract the budget from every beta's size — whoever goes below zero is deleted

$$\hat{\beta}_j^{\text{lasso}} = \text{sign}(b_j) \cdot \max(0, |b_j| - \lambda) \quad (\text{standardized, uncorrelated clues})$$

Why deletion happens

- The absolute-value penalty charges every beta the same 'entry fee' λ .
- A beta smaller than the fee cannot pay — it exits at exactly 0.
- Ridge's squared penalty charges tiny betas almost nothing, so they linger.

Lab guide — per λ

- One subtraction per clue: $|b| - \lambda$.
- Negative result $\rightarrow$ beta = 0 (fired). Positive $\rightarrow$ keep, restore the sign.
- Two passes ($\lambda = 0.25, 0.40$) over five rows.

Two Budget Rounds Through the Zoo

Each round: one subtraction per animal — the weak ones go to zero

Clue	OLS b	$ b - 0.25$	Lasso β ($\lambda=0.25$)	$ b - 0.40$	Lasso β ($\lambda=0.40$)
value	+0.90	0.65	+0.65	0.50	+0.50
momentum	+0.35	0.10	+0.10	-0.05 $\rightarrow$ 0	0 (fired)
size	+0.10	-0.15 $\rightarrow$ 0	0 (fired)	-0.30 $\rightarrow$ 0	0 (fired)
quality	-0.60	0.35	-0.35	0.20	-0.20
'lucky' factor	+0.05	-0.20 $\rightarrow$ 0	0 (fired)	-0.35 $\rightarrow$ 0	0 (fired)

$\lambda = 0.25$: 3 of 5 clues survive

$\lambda = 0.40$: 2 of 5 survive

ridge: all 5 shrink, none die

The Zoo After the Cull: $5 \rightarrow 3 \rightarrow 2$

Value and quality survive both rounds; the lucky factor never had a chance

$5 \rightarrow 3$

$\lambda = 0.25$ — size and the lucky factor are deleted

$3 \rightarrow 2$

$\lambda = 0.40$ — momentum falls next; value & quality remain

0.05

The lucky factor's beta — Ridge would have kept it at ~ 0.04

exactly 0

Only the L1 penalty produces true deletion

The zoo lesson

- Selection is a feature, not a side effect: fewer clues = auditable model.
- The survivors are the clues whose evidence beat the entry fee.

The honest caveat

- Real clues are correlated — Lasso then picks ONE cousin arbitrarily.
- That is exactly why Elastic Net exists (lecture, slide 21).

Scale It: A 30-Stock Tracking Basket

Paste the metaprompt — Claude Code lets Lasso build the index-tracking portfolio

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) downloads 5 years of daily returns for SPY plus 30 large-cap US stocks with yfinance;
(offline classroom: load `fml_w4_panel_offline.csv` and track the equal-weight average instead)
- 2) sets y = SPY returns, X = the 30 stocks, standardized inside a sklearn Pipeline;
- 3) sweeps Lasso alpha over a log grid; for each alpha records the number of stocks kept and the tracking error (std of $y - \hat{y}$) on a time-ordered test set;
- 4) plots kept-count and tracking error vs alpha, then prints the smallest basket within 1.5x the best error, with its tickers and weights;
- 5) three-sentence verdict: why Lasso (not Ridge) is the right tool here, in factor-zoo language.

[OUTPUT] What to expect

tight alphas: ~8-15 stocks
track SPY closely
tracking error rises gently
as the basket shrinks
kept names: the mega-caps
that ARE the index

Your 5-animal cull, run on
a 30-animal zoo.

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

OLS: slope = co-movement over variance

$\beta_1 = 26/20 = 1.30$ from four days — the market beta behind CAPM, hedging, risk budgets.

2

Ridge: λ in the denominator melts the beta

$1.30 \rightarrow 1.04 \rightarrow 0.65 \rightarrow 0.26$ — trust shrinks smoothly, but never to exactly zero.

3

Lasso: an entry fee that fires the weak

Soft threshold culled the zoo $5 \rightarrow 3 \rightarrow 2$ — the lucky factor died; value and quality survived.

Hand calculation first — then let Claude Code scale the same idea to real data.